

1. Introduction to Lexical Analysis

The lexical analyzer (or scanner) represents the first phase of a compiler. Its primary objective is to read the input source program character-by-character, group them into meaningful sequences called **lexemes**, and identify their grammatical categories (**tokens**). Concurrently, a lexical analyzer filters out extraneous components like blank spaces, tabulations, line endings, and user comments, shielding downstream compiler phases from parsing processing overhead.

2. Comprehensive Program Implementations

TASK 01: Whitespace Elimination and Number Collection

Objective: Write a C program to eliminate white spaces from a C source program stream and isolate integers or real numbers.

Algorithmic Strategy:

1. Scan the source text stream character-by-character.
2. If space (' '), tab ('\t'), or newline ('\n') is found, increment the pointer without printing, effectively stripping it.
3. If a digit is encountered, accumulate sequential digits into a buffer. If a single decimal point (.) is met, categorize the token as a floating-point number.

C Source Code:

```
Competitive Programming
Time.c  C dice.c  C pizzaCutting.c  C task01.c X  C loop01.c  C loop02.c  C loop03.c  C manyHelloWorld.c  C ageing.c  D  S  O
BasicProgram > CD_Lab > C task01.c > main()

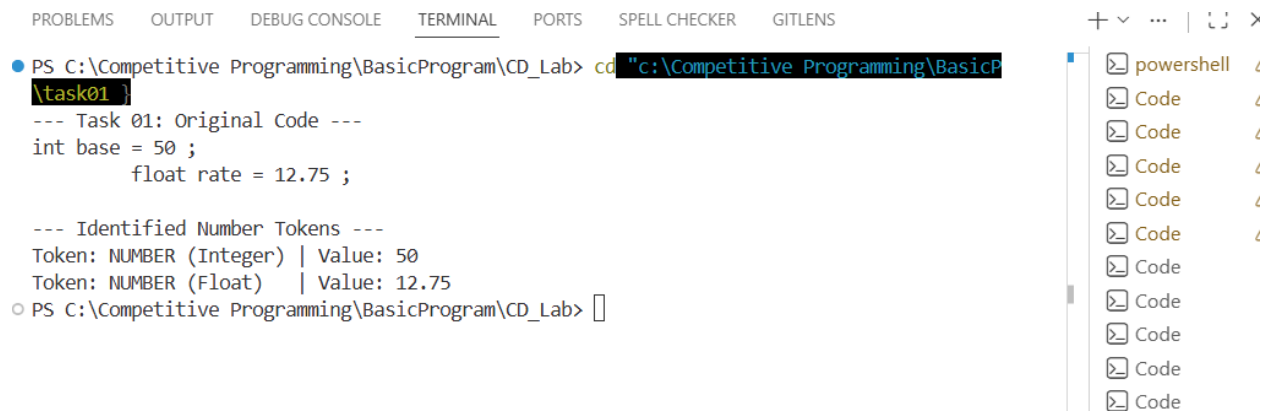
1  #include <stdio.h>
2  #include <ctype.h>
3  #include <string.h>
4  #include <stdbool.h>
5
6  bool isWhitespace(char ch)
7  {
8      return (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\r');
9  }
10
11 int main()
12 {
13     char sourceCode[] = "int base = 50 ; \n\t float rate = 12.75 ;";
14     int i = 0;
15     int length = strlen(sourceCode);
16
17     printf("--- Task 01: Original Code ---\n%s\n\n", sourceCode);
18     printf("--- Identified Number Tokens ---\n");
19
20     while (i < length)
21     {
22         if (isWhitespace(sourceCode[i]))
23         {
24             i++;
25             continue;
26         }
27
28         if (isdigit(sourceCode[i]))
29         {
30             char numBuffer[50];
31             int k = 0;
32             bool isFloat = false;
33
34             while (i < length && (isdigit(sourceCode[i]) || sourceCode[i] == '.'))
35             {

```

```
Competitive Programming
Time.c  C dice.c  X pizzaCutting.c  C task01.c u X  C loop01.c  C loop02.c  C loop03.c  C manyHelloWorld.c  C ageing.c  D v  [
BasicProgram > CD_Lab > C task01.c > main()

35     {
36         if (sourceCode[i] == '.')
37         {
38             if (isFloat)
39                 break;
40             isFloat = true;
41         }
42         numBuffer[k++] = sourceCode[i++];
43     }
44     numBuffer[k] = '\0';
45
46     if (isFloat)
47     {
48         printf("Token: NUMBER (Float) | Value: %s\n", numBuffer);
49     }
50     else
51     {
52         printf("Token: NUMBER (Integer) | Value: %s\n", numBuffer);
53     }
54 }
55 else
56 {
57     if (isalpha(sourceCode[i]))
58     {
59         while (i < length && isalnum(sourceCode[i]))
60             i++;
61     }
62     else
63     {
64         i++;
65     }
66 }
67 }
68 return 0;
```

Program Execution Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER GITLENS
PS C:\Competitive Programming\BasicProgram\CD_Lab> cd "c:\Competitive Programming\BasicP
\task01"
--- Task 01: Original Code ---
int base = 50 ;
    float rate = 12.75 ;

--- Identified Number Tokens ---
Token: NUMBER (Integer) | Value: 50
Token: NUMBER (Float) | Value: 12.75
PS C:\Competitive Programming\BasicProgram\CD_Lab> 
```

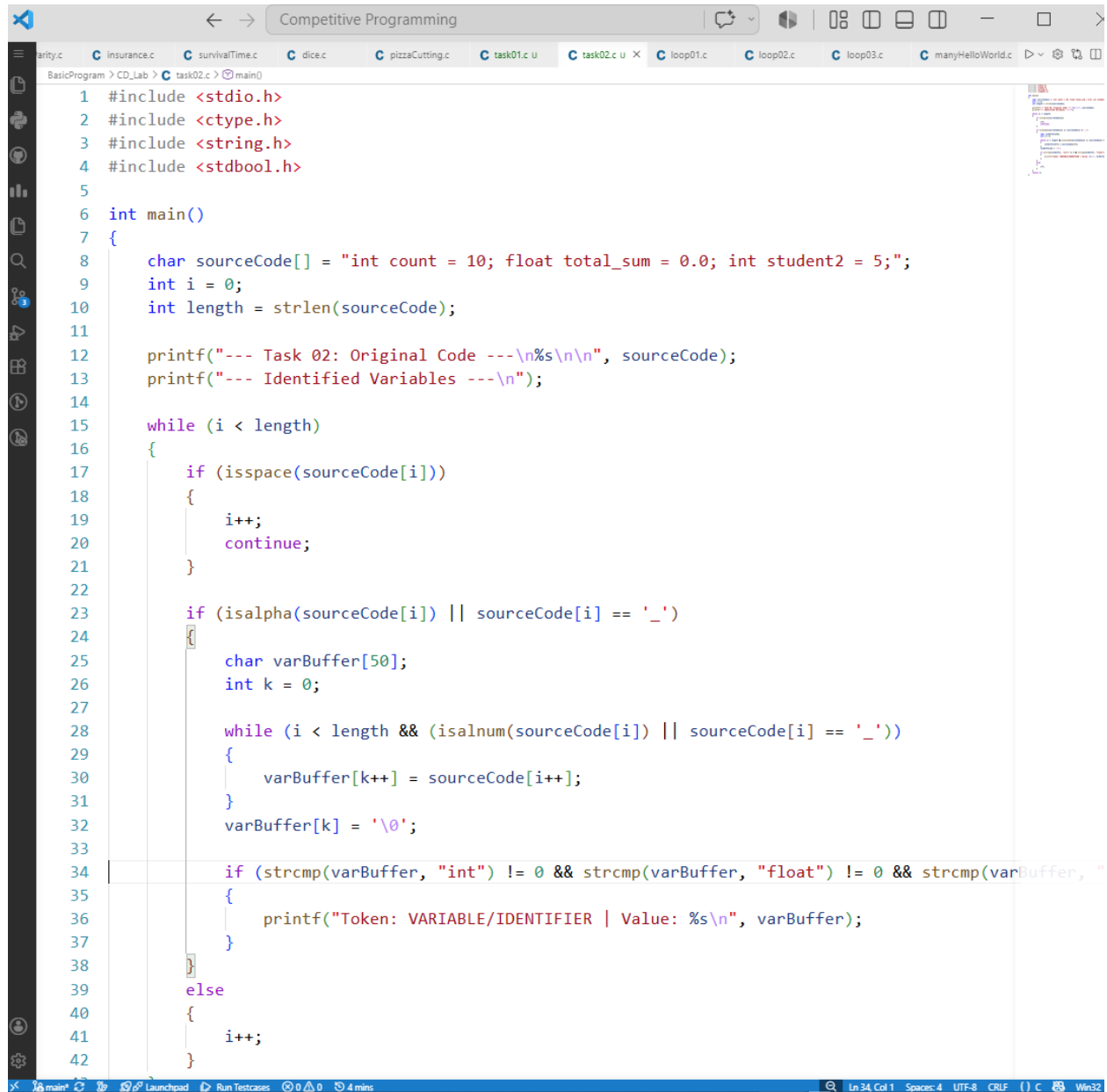
TASK 02: Variable / Identifier Recognition

Objective: Write a C program to recognize valid variables (identifiers) within an input text block based on standard C naming identifiers rules.

Algorithmic Strategy:

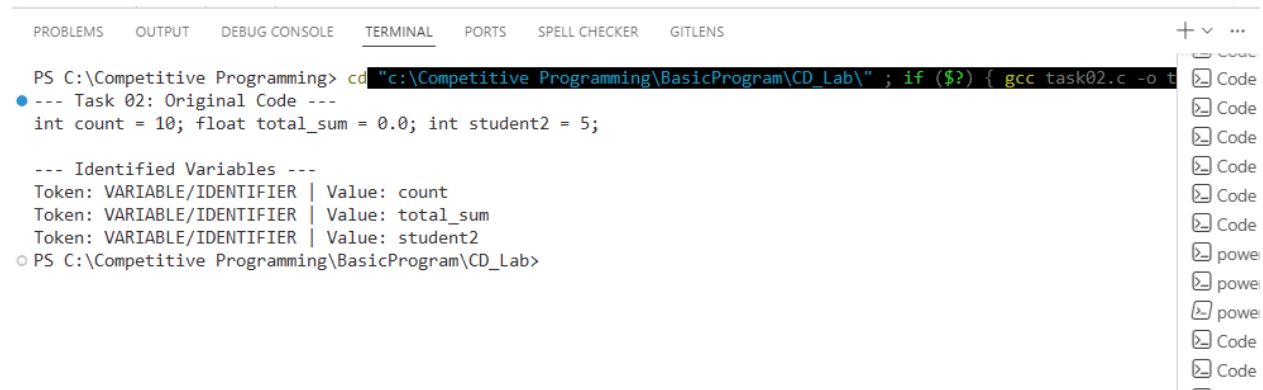
1. In C, a valid identifier must start with an alphabet letter (a-z, A-Z) or an underscore (_).
2. Subsequent characters can be letters, underscores, or digits (0-9).
3. Disregard explicit numbers or free-floating operators to cleanly print valid **variables**.

C Source Code:



```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 int main()
7 {
8     char sourceCode[] = "int count = 10; float total_sum = 0.0; int student2 = 5;";
9     int i = 0;
10    int length = strlen(sourceCode);
11
12    printf("--- Task 02: Original Code ---\n%s\n\n", sourceCode);
13    printf("--- Identified Variables ---\n");
14
15    while (i < length)
16    {
17        if (isspace(sourceCode[i]))
18        {
19            i++;
20            continue;
21        }
22
23        if (isalpha(sourceCode[i]) || sourceCode[i] == '_')
24        {
25            char varBuffer[50];
26            int k = 0;
27
28            while (i < length && (isalnum(sourceCode[i]) || sourceCode[i] == '_'))
29            {
30                varBuffer[k++] = sourceCode[i++];
31            }
32            varBuffer[k] = '\0';
33
34            if (strcmp(varBuffer, "int") != 0 && strcmp(varBuffer, "float") != 0 && strcmp(varBuffer, "
35            {
36                printf("Token: VARIABLE/IDENTIFIER | Value: %s\n", varBuffer);
37            }
38        }
39        else
40        {
41            i++;
42        }
43    }
```

Program Execution Output:



The screenshot shows a VS Code terminal window with the following content:

```
PS C:\Competitive Programming> cd "c:\Competitive Programming\BasicProgram\CD_Lab\" ; if ($?) { gcc task02.c -o t
• --- Task 02: Original Code ---
int count = 10; float total_sum = 0.0; int student2 = 5;

--- Identified Variables ---
Token: VARIABLE/IDENTIFIER | Value: count
Token: VARIABLE/IDENTIFIER | Value: total_sum
Token: VARIABLE/IDENTIFIER | Value: student2
○ PS C:\Competitive Programming\BasicProgram\CD_Lab>
```

On the right side of the terminal, there is a vertical list of icons. The first four are labeled 'Code' and the next four are labeled 'power'.

TASK 03: Keyword Recognition

Objective: Write a C program to recognize distinct, reserved keywords in a given source block.

Algorithmic Strategy:

1. Extract alphanumeric letter groups from the source program input stream.
2. Cross-reference the collected lexeme string against an array of C standard reserved system strings (like if, else, while, int, return).
3. Flag matches as an authenticated KEYWORD token.

C Source Code:

```
Competitive Programming
insurance.c survivalTime.c dice.c pizzaCutting.c task01.c U task02.c U task03.c U X loop01.c loop02.c li
BasicProgram > CD_Lab > C task03.c > checkKeyword(char [])
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 const char *keywords[] = {"int", "float", "char", "if", "else", "while", "for",
7 int numKeywords = 9;
8
9 bool checkKeyword(char buffer[])
10 {
11     for (int j = 0; j < numKeywords; j++)
12     {
13         if (strcmp(keywords[j], buffer) == 0)
14         {
15             return true;
16         }
17     }
18     return false;
19 }
20
21 int main()
22 {
23     char sourceCode[] = "if ( x > 0 ) return void ; else int y = 20 ;";
24     int i = 0;
25     int length = strlen(sourceCode);
26
27     printf("--- Task 03: Original Code ---\n%s\n\n", sourceCode);
28     printf("--- Identified Keywords ---\n");
29
30 main* Launchpad Run Testcases 0 0 0 4 mins
```

```

insurance.c  C survivalTime.c  C dice.c  C pizzaCutting.c  C task01.c U  C task02.c U  C task03.c U X  C loop01.c  C loop02.c  C loop03.c  C manyHello
BasicProgram > CD_Lab > C task03.c > checkKeyword(char [])
27     printf("--- Task 03: Original Code ---\n%s\n\n", sourceCode);
28     printf("--- Identified Keywords ---\n");
29
30     while (i < length)
31     {
32         if (isalpha(sourceCode[i]))
33         {
34             char buffer[50];
35             int k = 0;
36
37             while (i < length && isalnum(sourceCode[i]))
38             {
39                 buffer[k++] = sourceCode[i++];
40             }
41             buffer[k] = '\0';
42
43             if (checkKeyword(buffer))
44             {
45                 printf("Token: RESERVED KEYWORD      | Value: %s\n", buffer);
46             }
47         }
48         else
49         {
50             i++;
51         }
52     }
53     return 0;
54 }

```

Program Execution Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER  GIT LENS
● PS C:\Competitive Programming\BasicProgram\CD_Lab> cd "c:\Competitive Programming\BasicProgram\CD_Lab\"; if ($?) { g
--- Task 03: Original Code ---
if ( x > 0 ) return void ; else int y = 20 ;

--- Identified Keywords ---
Token: RESERVED KEYWORD | Value: if
Token: RESERVED KEYWORD | Value: return
Token: RESERVED KEYWORD | Value: void
Token: RESERVED KEYWORD | Value: else
Token: RESERVED KEYWORD | Value: int
○ PS C:\Competitive Programming\BasicProgram\CD_Lab>

```

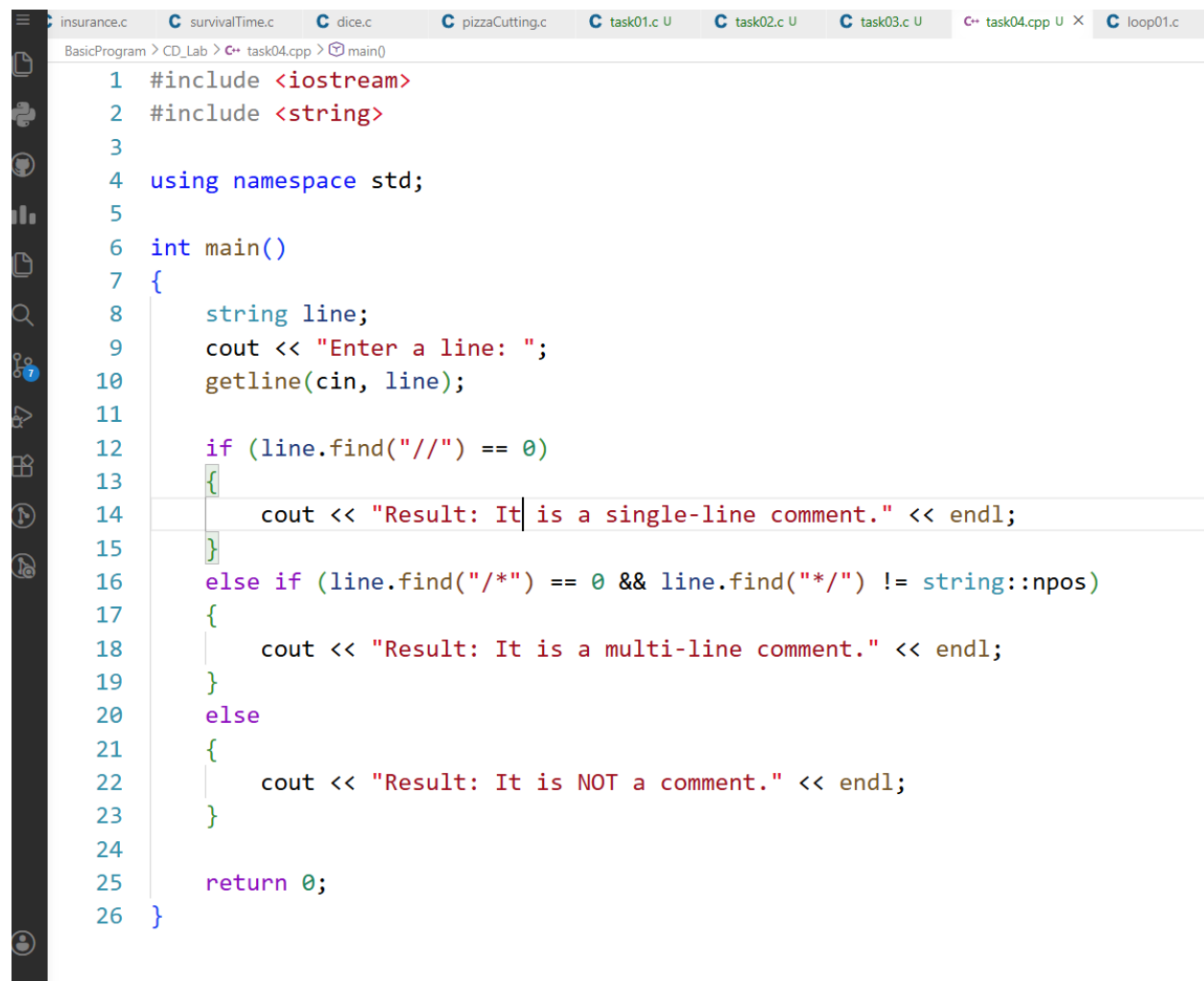
TASK 04: Finding Comments in a Program

Objective: Write a program to evaluate a string statement input and identify single-line or multi-line block comments.

Algorithmic Strategy:

1. Scan the prefix of a given line string input.
2. If a sequence starts explicitly with double forward slashes (//), parse it as a single-line block comment pattern.
3. If a line matches the starting sequence block marker (/*) and contains a terminating signature string (*/), track it as a complete multi-line comment token block.

C++ Source Code:



```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main()
7 {
8     string line;
9     cout << "Enter a line: ";
10    getline(cin, line);
11
12    if (line.find("//") == 0)
13    {
14        cout << "Result: It is a single-line comment." << endl;
15    }
16    else if (line.find("/*") == 0 && line.find("*/") != string::npos)
17    {
18        cout << "Result: It is a multi-line comment." << endl;
19    }
20    else
21    {
22        cout << "Result: It is NOT a comment." << endl;
23    }
24
25    return 0;
26 }
```

Program Execution Output:

```
Enter a line: // This is a tracking statement test
```

```
Result: It is a single-line comment.
```

```
Enter a line: /* multiline structure test buffer */
```

```
Result: It is a multi-line comment.
```

```
Enter a line: int data = 45;
```

```
Result: It is NOT a comment.
```

3. Discussion & General Conclusion

Across all four designated lab experiments, the core foundations of lexical token separation mechanics were effectively evaluated. By building simple character buffers and checking strings against explicit syntactic definitions, we successfully isolated functional code segments (identifiers, keywords, numbers) from ambient architectural layouts (comments and whitespace blocks). This procedural phase optimizes formatting overhead, guaranteeing an accurately filtered structural data collection stream before it moves to syntax parsing and semantic check trees.